PROJECT 3: SIEM DEPLOYMENT

Security Operations Report

Prepared by: [Your Name]

Registration Number: CYBE/2026/TC-7/0747

Class: Cyber Security Class B, Cohort 7

Team Members: CYBE/2026/TC-7/0590, CYBE/2026/TC-7/0452, CYBE/2026/TC-7/0747

Date:19, July 2026

Introduction

Our team was asked to act as a newly formed Security Operations Center (SOC) for an organization with no central visibility into its network. We were tasked with deploying a Security Information and Event Management (SIEM) solution from the ground up, using Wazuh, and proving it could collect logs from real systems and detect real attacks.

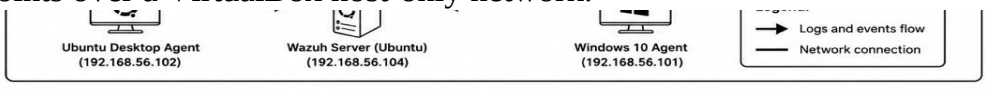
This report shows how we deployed the Wazuh SIEM, connected two endpoints (one Ubuntu Linux machine and one Windows 10 machine), simulated three different attacks against those endpoints, and analyzed the logs generated to confirm that our SIEM could detect them.

Objectives

- Deploy a central Wazuh Manager and connect endpoint agents to it
- Ingest Windows Event Logs and Linux syslogs/authentication logs
- Simulate at least three real-world attack vectors against our own endpoints
- Detect and analyze the logs and alerts generated by those attacks
- Produce a formal incident report for one of the simulated attacks

The architectural topology

The diagram below shows the full layout of our SIEM environment. The Wazuh Server, which runs the Manager, Indexer, and Dashboard, sits at the center and is connected to two endpoints over a VirtualBox host-only network.



This network topology clearly shows the Wazuh Server and both connected endpoints, with IP addresses

The Wazuh Server runs on Ubuntu Server at IP 192.168.56.104. It is connected to two endpoints: the Ubuntu Desktop agent at IP 192.168.56.102, and the Windows 10 agent at IP 192.168.56.101. Both endpoints send their logs and events to the server over the

192.168.56.1/24 network. Promiscuous mode is enabled on the network so the server can read traffic between the two endpoints.

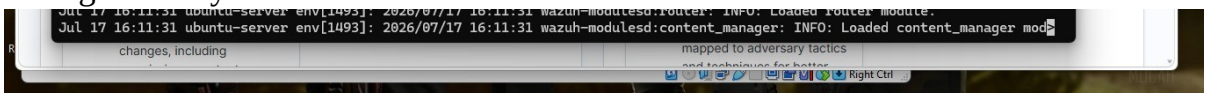
Infrastructure & log ingestion

This section shows how our team set up the Wazuh SIEM system and connected our two endpoints so their logs could be collected and viewed centrally.

The setup procedures

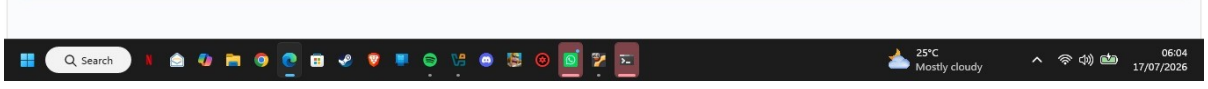
We used Oracle VirtualBox to create three virtual machines: one Ubuntu Server to act as the Wazuh Manager, one Ubuntu Desktop machine, and one Windows 10 machine. The last two acted as our endpoints.

We first installed the Wazuh Manager on the Ubuntu Server. After installation, we checked that the wazuh-manager service was running properly using the command sudo systemctl status wazuh-manager. The service showed as active and running, with all its core processes, such as wazuh-analysisd, wazuh-remoted, and wazuh-logcollector, working correctly.



Here,the terminal output confirmed that, the wazuh manager service is active and running Properly.

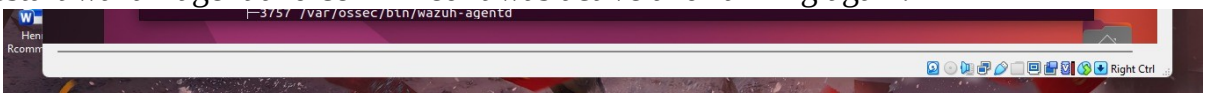
We then installed the Wazuh Agent on both endpoints and connected them to the Manager. After installation, we confirmed that both agents were active. The Wazuh Endpoints Summary dashboard showed 2 active agents: ubuntu-desktop-agent (ID 001, IP 192.168.56.102, running Ubuntu 22.04.5 LTS) and windows10-agent (ID 002, IP 192.168.56.101, running Windows 10 Pro). Both agents were on version 4.14.6 and connected to the same manager node.



The Wazuh Endpoints Summary dashboard showing both agents as Active

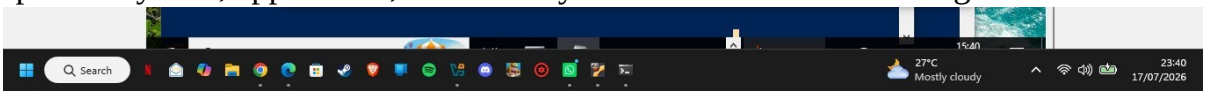
Telemetry configuration

On the Ubuntu endpoint, we edited the agent configuration file (ossec.conf) to collect the important log files. We added log sources for the authentication log (/var/log/auth.log), the system log (/var/log/syslog), the package installation log (/var/log/dpkg.log), and the active-responses log. After saving the file, we restarted the agent using sudo systemctl restart wazuh-agent and confirmed it was active and running again.



The diagram of the Ubuntu agent ossec.conf showing configured log sources, and the agent service running after restart

On the Windows endpoint, we edited the ossec.conf file to collect Windows Event Logs. We configured three log channels: Application, Security (with a filter to remove unimportant, noisy event IDs), and System. This allowed the Windows agent to forward important system, application, and security events to the Wazuh Manager.

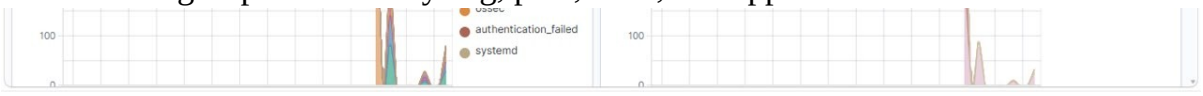


Here,Windows agent ossec.conf showing the application, Security, and System log channels configured

Proof of ingestion

After configuration, we confirmed that both endpoints were sending logs correctly by checking the Wazuh Threat Hunting dashboard.

For the Ubuntu endpoint (ubuntu-desktop-agent), the dashboard recorded 560 total alerts for the week, with 6 authentication failures and 50 successful authentications. The most common alert groups included syslog, pam, sudo, and apparmor.



Wazuh dashboard for ubuntu-desktop-agent showing total alerts and authentication activity

For the Windows endpoint (windows10-agent), the dashboard recorded 992 total alerts for the week, with 0 authentication failures and 6 successful authentications. The most

common alert groups included windows, windows_application, windows_security, and policy_changed.



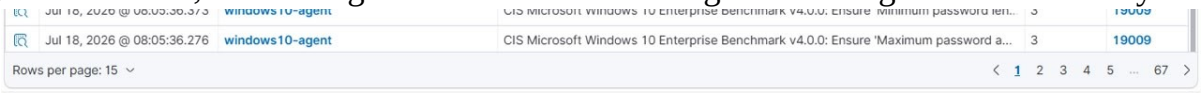
Wazuh dashboard for windows10-agent showing total alerts and authentication activity

We also checked the raw event logs for both agents under the Events tab. For the Ubuntu agent, we could see real log entries such as PAM login sessions, sudo attempts, and Apparmor denials, confirming that authentication logs and syslogs were being received correctly.



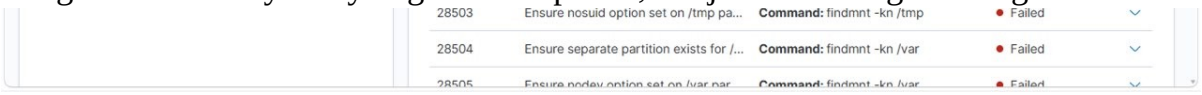
Here,you can clearly see raw event log entries received from the Ubuntu endpoint, including PAM and sudo activity

For the Windows agent, we could see entries related to CIS benchmark checks and system services, confirming that Windows Event Logs were being received correctly.



Raw event log entries received from the Windows endpoint, including CIS benchmark checks

Finally, we ran a Configuration Assessment (SCA) scan on both endpoints to check their security settings against CIS benchmarks. The Ubuntu endpoint scored 42% against the CIS Ubuntu Linux 22.04 LTS Benchmark (76 passed, 101 failed, 30 not applicable), and the Windows endpoint scored 27% against the CIS Microsoft Windows 10 Enterprise Benchmark (115 passed, 297 failed, 12 not applicable). This confirmed that the Wazuh Manager was actively analyzing both endpoints, not just collecting their logs.



CIS benchmark configuration assessment results for the Ubuntu endpoint



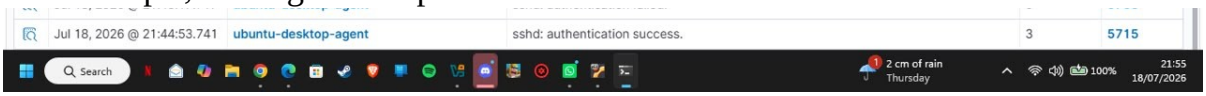
The CIS benchmark configuration assessment results for the Windows endpoint

Attack simulation

Once our baseline logging was confirmed, our team acted as the adversary and carried out three different attacks against our own endpoints, to check whether Wazuh could detect each one.

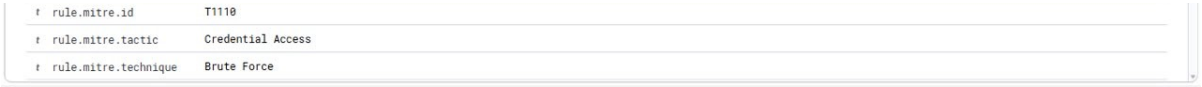
Attack 1: SSH brute-force (Linux endpoint)

We attempted to log in to the Ubuntu endpoint over SSH multiple times using an incorrect password, to simulate an attacker trying to guess valid credentials. After several failed attempts, one login attempt succeeded.



Terminal showing repeated failed SSH login attempts, followed by a successful login

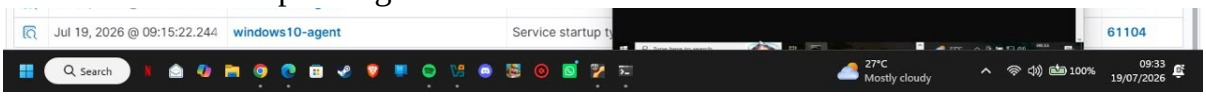
Wazuh detected the repeated failed logins and correctly classified the activity as a brute-force attempt, mapping it to MITRE ATT&CK technique T1110 (Brute Force) under the Credential Access tactic.



Wazuh alert detail showing the brute-force detection and its MITRE ATT&CK mapping

Attack 2: Hidden local administrator account creation (Windows endpoint)

On the Windows endpoint, we created a hidden local account (hiddenadmin1) using the command net user hiddenadmin1 p@ssw0rd123 /add, then added it to the local Administrators group using net localgroup Administrators hiddenadmin1 /add. This simulates an attacker planting a backdoor account for future access.



Command execution creating the hidden account and adding it to the Administrators group, with the resulting Wazuh alerts

Wazuh detected both actions: the creation of the account triggered an alert for 'User account enabled or created,' and adding it to the Administrators group triggered a higher-

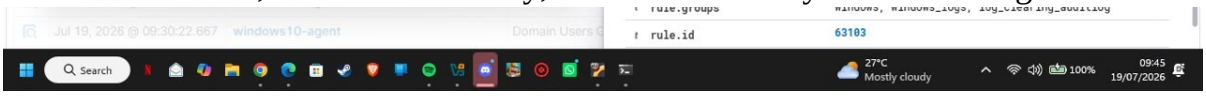
severity alert for 'Administrators Group Changed.' Wazuh mapped this to MITRE ATT&CK technique T1098 (Account Manipulation) under the Persistence tactic.



Wazuh alert detail showing the account manipulation detection and its MITRE ATT&CK mapping

5.3 Attack 3: Clearing Windows security event logs (Defense Evasion)

To simulate an attacker covering their tracks, we deleted the account we had created, then attempted to clear the Windows Security Event Log. Our first attempt used the clear command, which is not valid in Windows Command Prompt. We then used the correct Windows command, wevtutil cl security, which successfully cleared the log.



Terminal showing the account deletion and the command used to clear the Windows Security log

Even though the log was cleared, Wazuh still detected the clearing action itself, generating an alert stating 'The audit log was cleared,' along with the account name and computer name responsible for the action. This shows that Wazuh can catch defense evasion attempts, not just the original attack.



Wazuh alert detail confirming detection of the Security log being cleared

Detection engineering & formal incident report

All three simulated attacks were successfully detected by Wazuh. For a deeper, formal analysis, our team selected the Hidden Local Administrator Account attack, since it best demonstrates an attacker gaining a persistent foothold rather than just a single suspicious action.

Selected incident

Hidden Local Administrator Account Creation on the Windows 10 endpoint (windows10-agent).

MITRE ATT&CK mapping

Tactic: Persistence

Technique: T1098 - Account Manipulation

Raw log evidence

Wazuh recorded two key events for this incident. The raw log data for the first event, captured when the hiddenadmin1 account was created, is shown below:

```
rule.description: "User account enabled or created"
rule.id: 60109
rule.level: 8
rule.groups: windows, windows_security, adduser, account_changed
rule.mitre.id: T1098
rule.mitre.tactic: Persistence
rule.mitre.technique: Account Manipulation
manager.name: ubuntu-server
location: EventChannel
decoder.name: windows_eventchannel
```

The second event, generated when the account was added to the local Administrators group, was rule ID 60154 ("Administrators Group Changed"), severity level 12, recorded at 09:30:59 on July 19, 2026. Both raw events are also visible in the screenshots under Attack 2 above.

Attack timeline

09:29:00 - Attacker gains local access to the windows10-agent host (assumed foothold; no prior phishing or RDP step was simulated in this lab).

09:30:22 - Attacker runs net user hiddenadmin1 p@ssw0rd123 /add, creating a hidden local account. Wazuh generates Rule 60109 ("User account enabled or created," level 8).

09:30:59 - Attacker runs net localgroup Administrators hiddenadmin1 /add, elevating the account to local administrator. Wazuh generates Rule 60154 ("Administrators Group Changed," level 12).

Post-incident - If left undetected, the account would remain as a hidden, privileged backdoor available for future access. In our lab, the same session later included an attempt to remove the account and clear the Security log, which Wazuh also detected (see Attack 3 above).

Remediation recommendations

- Restrict local account creation: limit who can run net user and net localgroup by enforcing this through Group Policy, so only authorized administrators can create or modify accounts.
- Enable real-time alerting on Wazuh rules 60109 and 60154 so a SOC analyst is notified immediately when a new account is created or added to an administrator group.
- Regularly audit local Administrators group membership on all endpoints to catch unauthorized additions early.
- Protect Windows Event Logs from tampering, for example by forwarding logs to a separate, write-protected location, so clearing a local log does not destroy the evidence.
- Use an EDR or application whitelisting tool to block execution of account-management commands like net user and net localgroup by unauthorized users or processes.
- Require multi-factor authentication (MFA) for any action that changes administrator group membership.

Challenges and resolutions

During the SSH brute-force test, our first several login attempts returned "Permission denied" because we were not yet using the correct password for the target account. We resolved this by confirming the correct test credentials, after which the repeated failed attempts were correctly logged and later followed by a successful login, which triggered Wazuh's brute-force detection rule.

During the log-clearing test, our first command, clear, failed because it is a Linux/PowerShell command and is not recognized in the Windows Command Prompt. We resolved this by using the correct Windows equivalent, wevtutil cl security, which successfully cleared the Security log and was still picked up by Wazuh.

Conclusion

Our team successfully deployed a working Wazuh SIEM environment from scratch, connected one Linux and one Windows endpoint, and confirmed that logs from both systems were being collected and analyzed correctly. We simulated three different attack vectors: SSH brute-force, hidden administrator account creation, and Windows Security log clearing, and Wazuh successfully detected all three, correctly mapping two of them to their MITRE ATT&CK techniques.

This confirms that our SIEM deployment meets the goal set by the CISO: it can ingest data from company operating systems and reliably detect real-world attack behavior. One limitation we noted is that both endpoints scored low on their CIS benchmark scans (42% for Ubuntu, 27% for Windows), meaning the underlying systems still have many hardening gaps. Future work should focus on remediating these configuration weaknesses in addition to monitoring for attacks.